Open the evidence document **evidence.doc**

Make sure that your name, centre number and candidate number will appear on every page of this document. This document must contain your answers to each question.

Save this evidence document in your work area as:

evidence_ followed by your centre number_candidate number, for example: **evidence_zz999_9999**

A class declaration can be used to declare a record.

If the programming language used does **not** support arrays, a list can be used instead.

One source file is used to answer **Question 2**. The file is called **`Trees.txt`**

1 A program needs to take integer numbers as input, sort the numbers and then search for a specific number.

(a) The integer numbers will be stored in the global 1D array, `DataStored`, with space for up to 20 integers.

The global variable `NumberItems` stores the quantity of items the array contains.

Write program code to declare `DataStored` and `NumberItems`.

Save your program as **Question1_J24**. Copy and paste the program code into part **1(a)** in the evidence document.

[1]

(b) The procedure `Initialise()`:

- prompts the user to input the quantity of numbers the user would like to enter
- reads the input and validates it is between 1 and 20 (inclusive)
- prompts the user to input each number and stores each number in `DataStored`.

Write program code for `Initialise()`.

Save your program. Copy and paste the program code into part **1(b)** in the evidence document.

[5]

© UCLES 2024

(c) The main program stores `0` in `NumberItems`, calls `Initialise()` and then outputs the contents of `DataStored`.

(i) Write program code for the main program.

Save your program.

Copy and paste the program code into part **1(c)(i)** in the evidence document.

[2]

(ii) Test your program by inputting the following data in the order given:

30

5

3

9

4

1

2

Take a screenshot of the output(s).

Save your program.

Copy and paste the screenshot into part **1(c)(ii)** in the evidence document.

[2]

(d) The procedure `BubbleSort()` uses a bubble sort to sort the data in `DataStored` into ascending numerical order.

(i) Write program code for `BubbleSort()`.

Save your program.

Copy and paste the program code into part **1(d)(i)** in the evidence document.

[4]

(ii) Write program code to amend the main program to call `BubbleSort()` and then output the contents of `DataStored`.

Save your program.

Copy and paste the program code into part **1(d)(ii)** in the evidence document.

[1]

(iii) Test your program by inputting the following data in the order given:

5

3

9

4

1

2

Take a screenshot of the output(s).

Save your program. Copy and paste the screenshot into part **1(d)(iii)** in the evidence document.

[1]

(e) The function `BinarySearch()`:

- takes the integer parameter `DataToFind` to search for in the array
- performs an iterative binary search on the array `DataStored`
- returns the index where `DataToFind` is found in `DataStored`. If `DataToFind` is **not** found, the function returns `-1`.

(i) Write program code for the iterative function `BinarySearch()`.

Save your program. Copy and paste the program code into part **1(e)(i)** in the evidence document.

[6]

(ii) Write program code to amend the main program to:

- take a number as input from the user
- call `BinarySearch()` with the number input
- output the value returned from the function call as its parameter.

Save your program. Copy and paste the program code into part **1(e)(ii)** in the evidence document.

[3]

(iii) Test your program twice with the following inputs:

Test 1: 5 1 6 2 8 10 2

Test 2: 5 1 6 2 8 10 7

Take a screenshot of the output(s).

Save your program.

Copy and paste the screenshot into part **1(e)(iii)** in the evidence document.

[2]